

Azure Deployment Guide

This guide describes how to deploy the containerized AI Investment Research System on Microsoft Azure. We cover two deployment methods:

1. **Azure App Service (Web App for Containers)** - Simple, standard web hosting.
2. **Azure Container Apps (ACA)** - Modern serverless container hosting (recommends scaling to zero when idle to save costs).



Prerequisites

1. **Azure CLI:** Install the Azure CLI from [here](#).
2. **Azure Account:** Run `az login` to log in to your active Azure subscription.
3. **Environment Variables:** Collect the keys from your local `.env` file (e.g. `GROQ_API_KEY`, `TAVILY_API_KEY`, etc.).



Step 1: Resource Setup

Open your terminal (PowerShell, Bash, or Command Prompt) and run the following commands:

```
# 1. Define variables (change names as needed)
RESOURCE_GROUP="InvestmentResearchRG"
LOCATION="eastus"
ACR_NAME="invresearchregistry" # Must be globally unique, alphanumeric only
APP_NAME="ai-investment-research" # Must be globally unique

# 2. Create a Resource Group
az group create --name $RESOURCE_GROUP --location $LOCATION

# 3. Create an Azure Container Registry (ACR)
az acr create --resource-group $RESOURCE_GROUP --name $ACR_NAME --sku Basic
```



Step 2: Build and Push Docker Image

You can build and push the image directly in the cloud using Azure Container Registry Tasks. **This means you do not even need Docker installed locally!**

Run this command from the root of your project directory:

```
az acr build --registry $ACR_NAME --image agents-demo:latest .
```



Step 3: Deploying the Container

Choose one of the two hosting options below.

Option A: Azure App Service (Web App for Containers)

App Service is standard and easy to configure.

```
# 1. Create a Linux App Service Plan (B1 basic plan shown here)
```

```
az appservice plan create \  
  --name "InvestmentResearchPlan" \  
  --resource-group $RESOURCE_GROUP \  
  --location $LOCATION \  
  --sku B1 \  
  --is-linux
```

```
# 2. Enable admin credentials on ACR (required for App Service authentication)
```

```
az acr update --name $ACR_NAME --admin-enabled true
```

```
# 3. Retrieve ACR username and password
```

```
ACR_USERNAME=$(az acr credential show --name $ACR_NAME --query "username" -o tsv)  
ACR_PASSWORD=$(az acr credential show --name $ACR_NAME --query "passwords[0].value" -
```

```
# 4. Create the Web App with the Docker container
```

```
az webapp create \  
  --resource-group $RESOURCE_GROUP \  
  --plan "InvestmentResearchPlan" \  
  --name $APP_NAME \  
  --deployment-container-image-name "$ACR_NAME.azurecr.io/agents-demo:latest" \  
  --docker-registry-server-user $ACR_USERNAME \  
  --docker-registry-server-password $ACR_PASSWORD
```

```
# 5. Route App Service traffic to Streamlit port (8501)
az webapp config appsettings set \
  --resource-group $RESOURCE_GROUP \
  --name $APP_NAME \
  --settings WEBSITES_PORT_BITMASK=8501 PORT=8501
```

Option B: Azure Container Apps (Recommended & Serverless)

Azure Container Apps scales automatically and can scale down to 0 instances when not in use to save cost.

```
# 1. Register the Container Apps extension in CLI
az extension add --name containerapp --upgrade

# 2. Create the Container App Environment
ENV_NAME="InvestmentResearchEnv"
az containerapp env create \
  --name $ENV_NAME \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION

# 3. Create the Container App
az containerapp create \
  --name $APP_NAME \
  --resource-group $RESOURCE_GROUP \
  --environment $ENV_NAME \
  --image "$ACR_NAME.azurecr.io/agents-demo:latest" \
  --target-port 8501 \
  --ingress 'external' \
  --registry-server "$ACR_NAME.azurecr.io"
```



Step 4: Configure App Settings (API Keys)

You need to feed your LLM and tool API credentials to the container.

For Option A (Azure App Service):

```
az webapp config appsettings set \
  --resource-group $RESOURCE_GROUP \
  --name $APP_NAME \
  --settings \
    GROQ_API_KEY="your-groq-api-key" \
    TAVILY_API_KEY="your-tavily-api-key" \
    LANGCHAIN_API_KEY="your-langchain-api-key" \
    LANGCHAIN_TRACING_V2="true" \
    LANGCHAIN_PROJECT="Investment_summarizer"
```

For Option B (Azure Container Apps):

```
az containerapp update \
  --name $APP_NAME \
  --resource-group $RESOURCE_GROUP \
  --set-env-vars \
    GROQ_API_KEY="your-groq-api-key" \
    TAVILY_API_KEY="your-tavily-api-key" \
    LANGCHAIN_API_KEY="your-langchain-api-key" \
    LANGCHAIN_TRACING_V2="true" \
    LANGCHAIN_PROJECT="Investment_summarizer"
```

Step 5: Verification

1. Get the URL of your deployed application:

- **App Service:** Run `az webapp show --name $APP_NAME --resource-group $RESOURCE_GROUP --query "defaultHostName" -o tsv`
- **Container App:** Run `az containerapp show --name $APP_NAME --resource-group $RESOURCE_GROUP --query "properties.configuration.ingress.fqdn" -o tsv`

2. Open the URL in your browser.

3. Trigger an analysis for an Indian equity (e.g. **SBI** or **Reliance**) to verify the Streamlit frontend communicates successfully with the FastAPI backend inside the container.